

Arquitectura Relacional y Consultas Multitabla

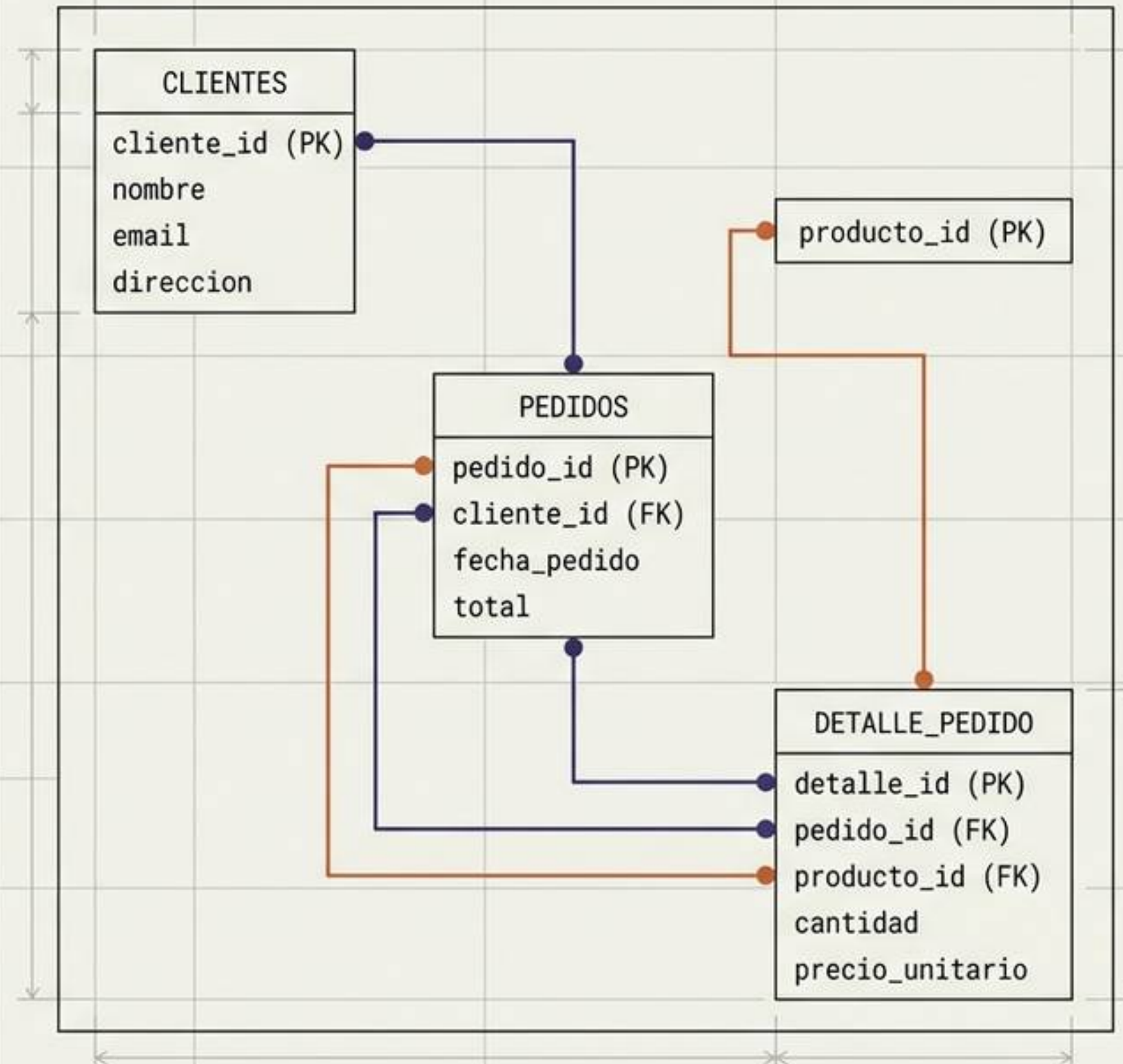
El enfoque de ingeniería para reconstruir información distribuida.



PRIMARY_KEY



FOREIGN_KEY



El problema: La normalización aísla los datos

id_venta	id_cliente	fecha_venta
-----	-----	-----
101	3	15/09/2026
102	1	16/09/2026

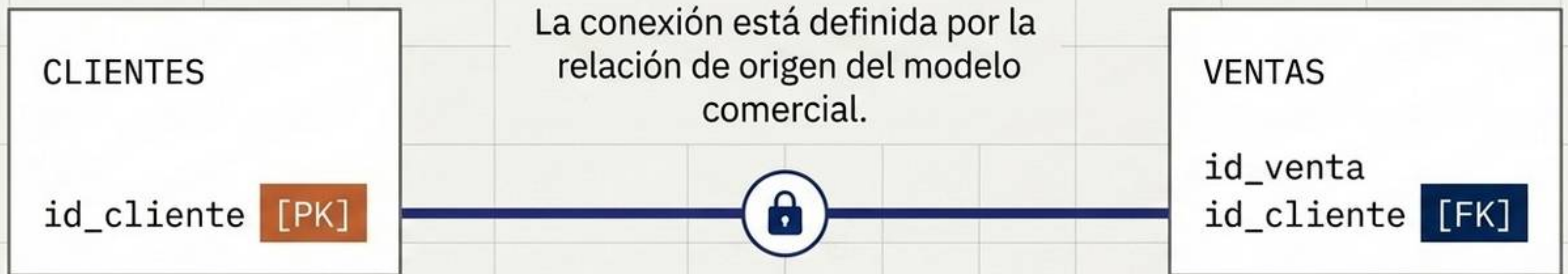
El usuario comercial no quiere saber que `id_cliente = 3`. Quiere saber:
¿Quién realizó la venta?

CLIENTES

`id_cliente` [PK]
nombre
correo
ciudad

La respuesta se encuentra en otra tabla,
físicamente separada por el diseño.

El Mecanismo: Las llaves actúan como puentes lógicos



El modelamiento relacional tiene una consecuencia práctica directa: reconstruir la realidad.

Primary Key (PK) ↔ Foreign Key (FK)

Anatomía de la sintaxis INNER JOIN

Qué información
queremos ver.

El comando de unión. Solo
muestra registros que poseen
coincidencia en ambas tablas.

```
SELECT columnas  
FROM tabla1  
INNER JOIN tabla2  
ON tabla1.campo = tabla2.campo;
```

La tabla base o
punto de partida.

La regla de coincidencia
(El puente lógico PK = FK).

Refinando la técnica: Uso de Alias

Sintaxis Densa



```
SELECT clientes.nombre,  
       ventas.id_venta,  
       ventas.fecha_venta  
FROM clientes  
INNER JOIN ventas  
    ON clientes.id_cliente =  
       ventas.id_cliente;
```



Estándar de la Industria



```
SELECT c.nombre,  
       v.id_venta,  
       v.fecha_venta  
FROM clientes c  
INNER JOIN ventas v  
    ON c.id_cliente = v.id_cliente;
```

Los alias no cambian los datos ni las tablas. Son nombres temporales utilizados para simplificar y hacer más legible la consulta.

c → CLIENTES
v → VENTAS

La expansión de la granularidad (Relación 1:N)

VENTAS		
id_venta: 101	cliente: Ana Pérez	fecha: 15/09/2026

¿Por qué una misma venta aparece varias veces tras un JOIN?

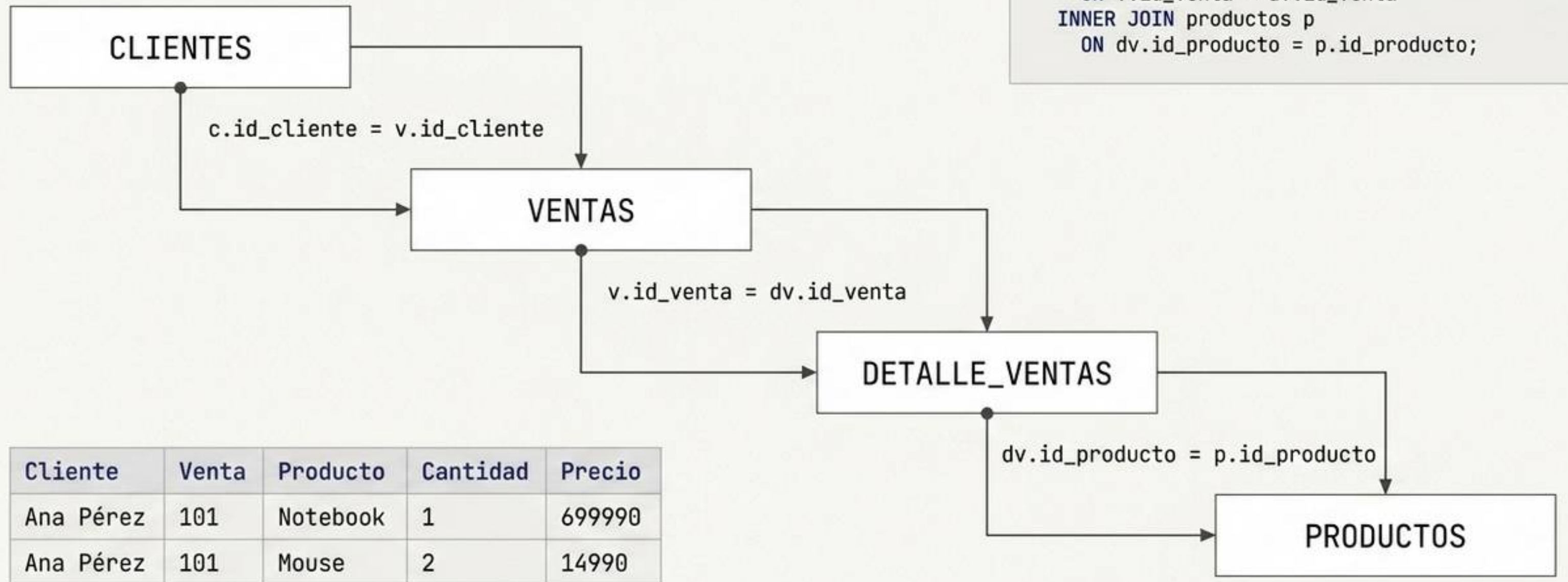
Porque una venta puede contener varios productos. El JOIN no duplica incorrectamente la venta; simplemente revela la granularidad del detalle.

DETALLE_VENTAS		
id_venta: 101	producto: Notebook	cant: 1 precio: 699990

DETALLE_VENTAS		
id_venta: 101	producto: Mouse	cant: 2 precio: 14990

Escalando la solución: JOIN Múltiple

A partir de cuatro tablas independientes, reconstruimos una visión de negocio totalmente comprensible.

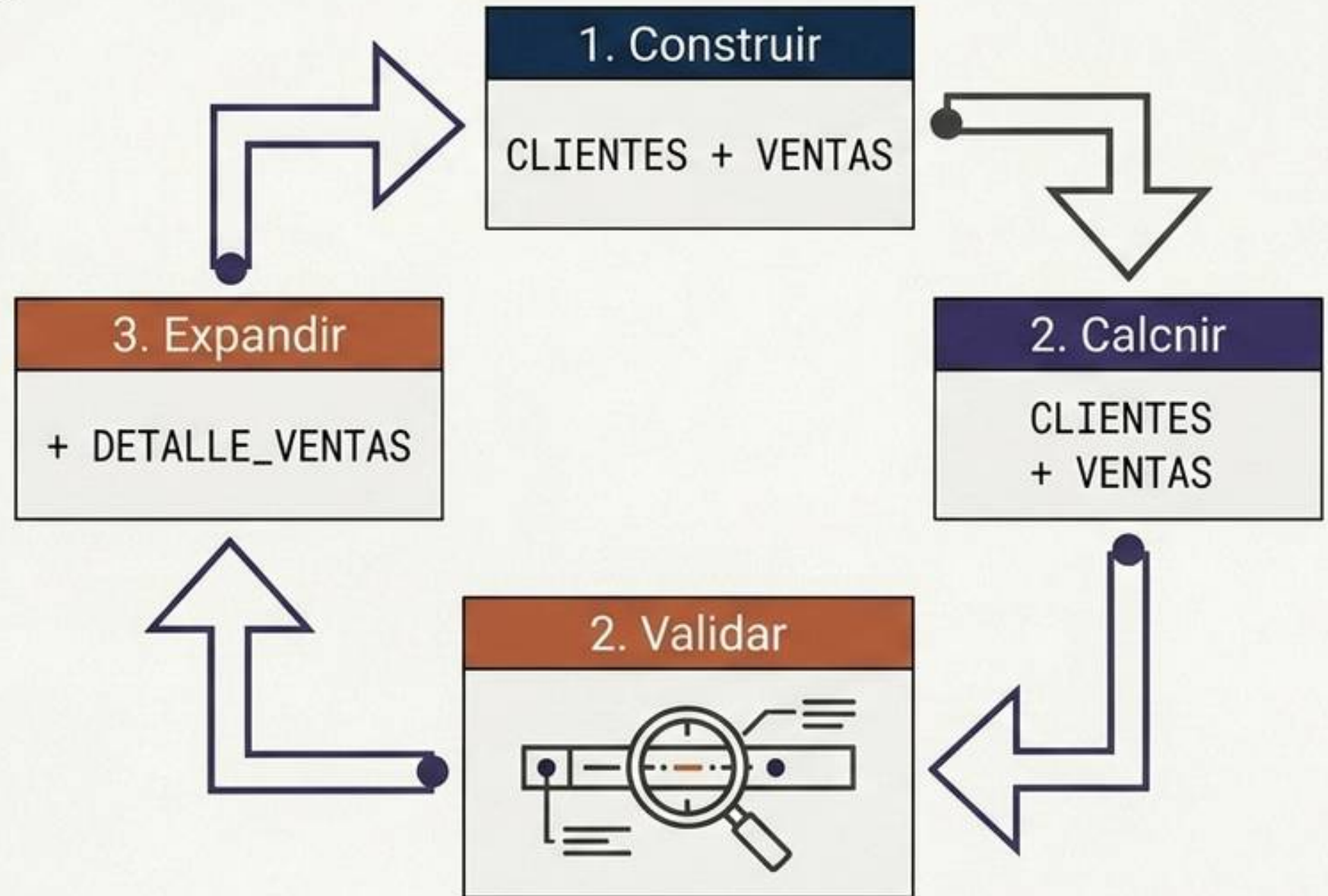


Regla de Oro: Construcción Progresiva

Nunca construyas una consulta de cuatro tablas de golpe.

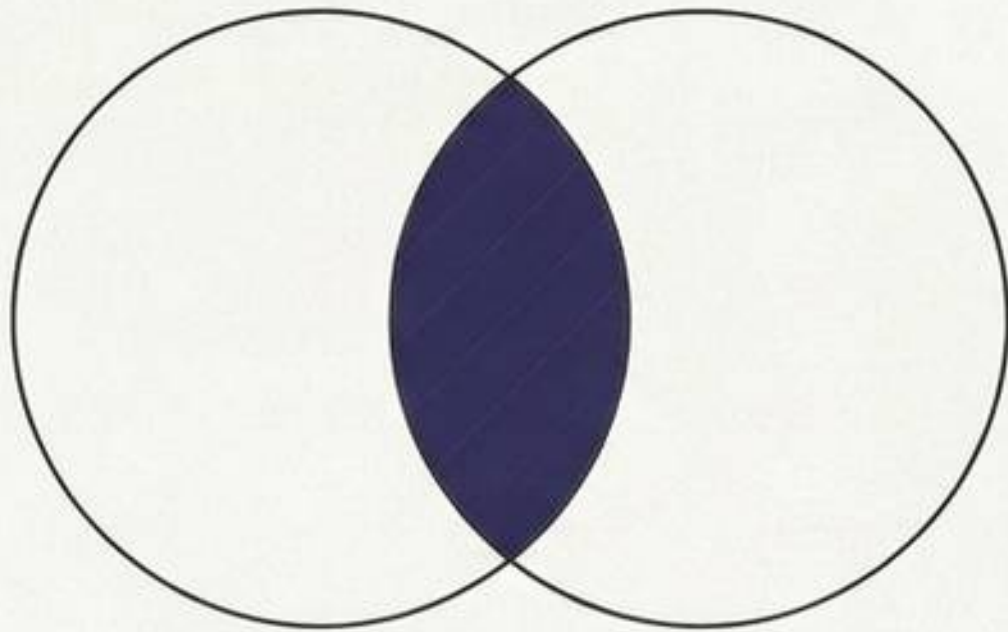
Prevención de Errores

- Permite detectar relaciones incorrectas.
- Previene el uso de columnas equivocadas.
- Evita duplicidades imprevistas.
- Aísla los errores de sintaxis en el ON.



Comportamiento de intersección: INNER vs LEFT JOIN

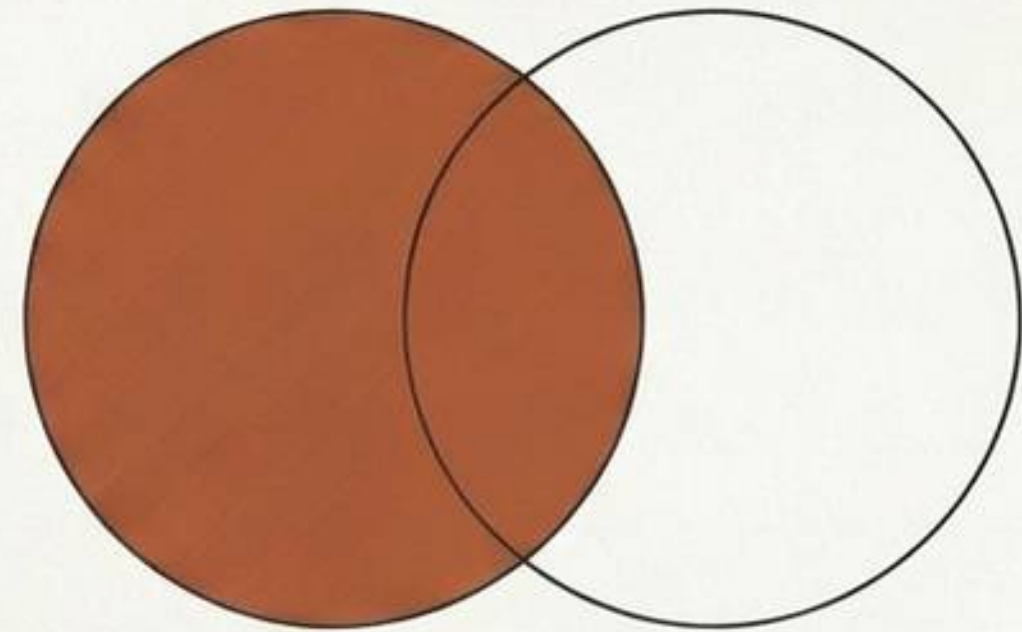
INNER JOIN



Solo coincidencias.

CLIENTE	ID_VENTA
Ana Pérez	101
Carlos Soto	102

LEFT JOIN



Todos los de la izquierda + coincidencias.

CLIENTE	ID_VENTA
Ana Pérez	101
Carlos Soto	102
Pedro Díaz	NULL

Detectando Ausencias en el Negocio

¿Qué ocurre si queremos identificar a los clientes que NUNCA han realizado una compra?

```
SELECT c.nombre,  
       v.id_venta  
FROM clientes c  
LEFT JOIN ventas v  
  ON c.id_cliente = v.id_cliente  
WHERE v.id_venta IS NULL;
```



Pedro Díaz | NULL

Al combinar LEFT JOIN con IS NULL, transformamos la falta de datos en información de valor.

Esto no es un error informático, es un descubrimiento analítico.

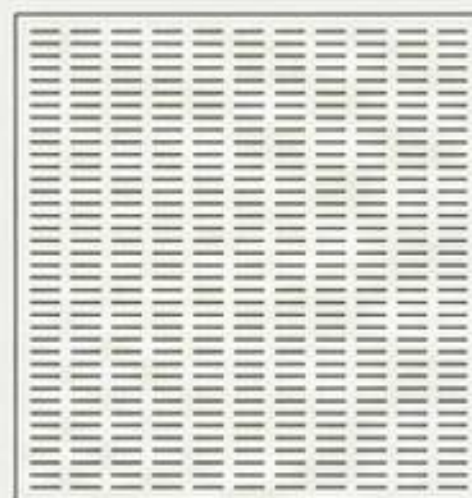
Del Detalle al Indicador

Leyes del Modelo Relacional

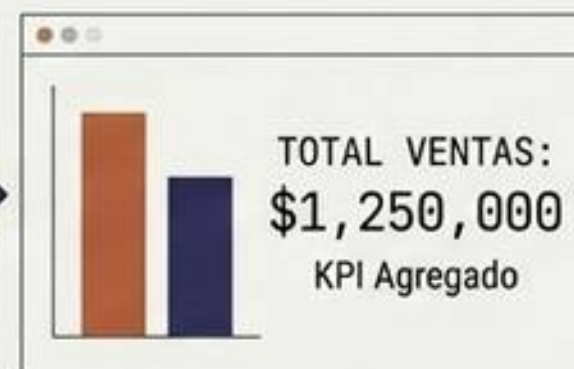
El JOIN depende estrictamente del modelo: siempre PK ↔ FK.

La granularidad de las tablas involucradas determina el volumen total de filas.

El próximo desafío



Datos Brutos:
Cientos de Transacciones



- Reconstruimos las transacciones línea por línea.
- Pero la gerencia requiere totales, promedios y máximos.
- Siguiente paso: Agregaciones (**COUNT**, **SUM**, **GROUP BY**).